```
public void insert(String word) {
    Node curr = root;
    for (int i = 0; i < word.length(); i++) {
      char ch = word.charAt(i);
      if (curr.child.containsKey(ch)) {
        curr = curr.child.get(ch);
      } else {
        Node nn = new Node(ch);
        curr.child.put(ch, nn);
        curr = nn;
      }
    }
    curr.isterminal = true;
  }
```

Sample Input

```
4
add hack
add hackerrank
find hac
find hak
```

Sample Output

```
2
0
```

```
public static int oranges_Rotting(int[][] grid) {
        Queue<int[]> q = new LinkedList<>();
        int fresh_orange = 0;
        for (int i = 0; i < grid.length; i++) {
            for (int j = 0; j < grid[0].length; j++) {
                if (grid[i][j] == 2) {
                    q.add(new int[] { i, j });
                } else if (grid[i][j] == 1) {
                    fresh_orange++;
                }
            }
        }
        System.out.println(q.size());
        int[] r = { -1, 0, 1, 0 };
        int[] c = { 0, -1, 0, 1 };
        int minutes = 0;
        while (!q.isEmpty()) {
            int size = q.size();
            for (int i = 0; i < size; i++) {
                int[] a = q.poll();
                for (int j = 0; j < 4; j++) {
                    int new_row = a[0] + r[j];
                    int new_col = a[1] + c[j];
                    if (new_row >= 0 && new_col >= 0 && new_row < grid.length && new_col < grid[0].length
                            && grid[new_row][new_col] == 1) {
                        grid[new_row][new_col] = 2;
                        fresh_orange--;
                        q.add(new int[] { new_row, new_col });
                    }
                }
            }
            System.out.println(q.size());
            minutes++;
        }
        System.out.println(minutes);
        return fresh_orange > 0 ? -1 : minutes;
    }
```

```
public static int oranges_Rotting(int[][] grid) {
        Queue<int[]> q = new LinkedList<>();
        int fresh_orange = 0;
        for (int i = 0; i < grid.length; i++) {
            for (int j = 0; j < grid[0].length; j++) {
                if (grid[i][j] == 2) {
                    q.add(new int[] { i, j });
                } else if (grid[i][j] == 1) {
                    fresh_orange++;
                }
            }
        }
        System.out.println(q.size());
        int[] r = { -1, 0, 1, 0 };
        int[] c = { 0, -1, 0, 1 };
        int minutes = 0;
        while (!q.isEmpty()) {
            int size = q.size();
            for (int i = 0; i < size; i++) {
                int[] a = q.poll();
                for (int j = 0; j < 4; j++) {
                    int new_row = a[0] + r[j];
                    int new_col = a[1] + c[j];
                    if (new_row >= 0 && new_col >= 0 && new_row < grid.length && new_col < grid[0].length
                            && grid[new_row][new_col] == 1) {
                        grid[new_row][new_col] = 2;
                        fresh_orange--;
                        q.add(new int[] { new_row, new_col });
                    }
                }
            }
            System.out.println(q.size());
            minutes++;
        }
        System.out.println(minutes);
        return fresh_orange > 0 ? -1 : minutes;
    }
```

```
class Trie {
    class Node {
        char ch;
        String isterminal;
        HashMap<Character, Node> child;

        public Node(char ch) {
            // TODO Auto-generated constructor stub
            this.ch = ch;
            child = new HashMap<>();
        }
    }

    private Node root;

    public Trie() {
        // TODO Auto-generated constructor stub
        root = new Node('*');
    }

    public void insert(String word) {
        Node curr = root;
        for (int i = 0; i < word.length(); i++) {
            char ch = word.charAt(i);
            if (curr.child.containsKey(ch)) {
                curr = curr.child.get(ch);
            } else {
                Node nn = new Node(ch);
                curr.child.put(ch, nn);
                curr = nn;
            }
        }
        curr.isterminal = word;
    }

    public String replaced(String s) {
        Node curr = root;
        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);
            if (curr.child.containsKey(ch)) {
                curr = curr.child.get(ch);
                if (curr.isterminal != null) {
                    return curr.isterminal;
                }
            } else {
                return s;
            }
        }
        return s;
    }
}

class Solution {
    public String replaceWords(List<String> dictionary, String sentence) {
        String[] arr = sentence.split(" ");
        Trie t = new Trie();
        for (String s : dictionary) {
            t.insert(s);
        }
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < arr.length; i++) {
            String s = t.replaced(arr[i]);
            sb.append(s + " ");
        }
        return sb.toString().trim();
    }
}
```